ANALYZING AND STUDYING SOFTWARE REQUIREMENTS, BOTH FUNCTIONAL AND NON-FUNCTIONAL

SECOND-WEEK ASSIGNMENT FOR THE SOFTWARE ENGINEERING COURSE

Lecturer :

Dimas Novian Aditia Syahputra, S.Tr.T., M.Tr.T.

Presented by :

Aisyah Currents

25091397108 / 2025I

INFORMATIC MANAGEMENT MAJOR

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2026

I. Definition of Software Requirements

Software Requirements is a description of requirements or specifications that explains what a software system must possess and perform in order to meet user needs. It serves as the foundation in the software development process because it functions as a guideline for developers in designing, creating, and testing the system. Software requirements form the cornerstone of effective system development, outlining what a software system must achieve and how it should perform to satisfy user needs. These requirements are typically categorized into two primary types: functional requirements, which define the specific behaviors and features of the system, and non-functional requirements, which address the quality attributes and operational standards. This distinction is crucial as it provides developers with clear guidelines for designing, implementing, and testing the software, ensuring alignment with business objectives and user expectations. By establishing these requirements early, potential issues such as scope creep, performance bottlenecks, or usability flaws can be mitigated, leading to a more robust and efficient final product.

II. Functional Requirements

Covers every activity from requirements elicitation, specification, architecture, design, coding, testing, deployment, operation, maintenance, to final retirement. Functional requirements specify the exact functionalities that the software must deliver, focusing on the "what" of the system namely, the actions, processes, and services it provides to users. These requirements are directly tied to user interactions and business processes, describing inputs, outputs, and the logic governing system operations. They are typically verifiable through testing scenarios that check whether the intended features operate as specified, making them essential for defining the core capabilities of the application.

In practice, functional requirements outline the system's behaviors in response to user actions or events. For instance, in an inventory management system, these might include allowing users to log in securely, enabling administrators to add or update item details, permitting staff to record incoming and outgoing stock transactions, generating inventory reports on demand, and maintaining supplier information. Such requirements ensure that the software supports key workflows, such as data entry, retrieval, and processing, without ambiguity. By prioritizing these, developers can build a system that

directly addresses operational needs, fostering productivity and reducing manual errors in day-to-day tasks.

III. Non-Functional Requirements

Non-functional requirements, in contrast, emphasize the "how" of the system, detailing the qualitative attributes that influence its overall performance, reliability, and user experience. These are not about specific features but rather the standards of excellence that the software must uphold, including aspects like speed, security, accessibility, and maintainability. Unlike functional requirements, they are often harder to test directly and may involve metrics or benchmarks to evaluate compliance, ensuring the system not only works but works well under various conditions.

Examples of non-functional requirements include stipulating that the system responds to user requests within three seconds to maintain efficiency, implementing robust authentication mechanisms to protect sensitive data, supporting compatibility across desktop and mobile browsers for broader accessibility, guaranteeing 24/7 availability with minimal downtime, and incorporating secure data storage with regular backups to prevent loss. These elements are vital for long-term success, as they address user satisfaction, regulatory compliance, and scalability. For example, poor performance could lead to user frustration, while inadequate security might expose the system to breaches, underscoring the need to integrate these requirements from the outset to avoid costly revisions later.

IV. Key Differences and Implications

The primary distinction between functional and non-functional requirements lies in their scope: functional ones drive the development of tangible features and user-facing capabilities, while non-functional ones set the benchmarks for system quality and sustainability. Functional requirements might dictate that a system "adds a new inventory item," whereas non-functional ones ensure it does so "quickly and securely without errors." This separation allows for a balanced approach in development, where features are built to function correctly (functional) and then refined to meet performance and usability thresholds (non-functional).

To illustrate simply, functional requirements are akin to the menu items in a restaurant specifying what dishes are available meanwhile non-functional requirements

cover the quality of service, such as the taste, speed of delivery, and ambiance. Together, they ensure a comprehensive blueprint for software that is both feature-rich and reliable. Neglecting either can result in a system that either lacks essential tools or fails to deliver them effectively, highlighting their interdependent role in the software lifecycle.